

2026 春数据结构与算法期末试卷（回忆版）参考答案

免责声明

本参考答案由 Codex 根据当前整理后的题面独立编写，仅供期末复习参考，不保证与原试卷标准答案或任课教师给出的评分标准完全一致。

答案速览

选择题速览

1. D 2. B 3. A 4. C 5. B
6. B 7. C 8. A 9. B 10. C

填空题速览

1. 24
2. 82
3. $O(\sqrt{n})$
4. 例如: a, b, d, e, f, c
5. $2^{k-1} \leq n \leq 2^k - 1$
6. 长度为 3, 深度为 3
7. 一条可行路径: (4,3) -> (3,3) -> (3,2) -> (2,2) -> (1,2)

大题结论速览

1. 二叉排序树构造完成后, $ASL_{\text{成}} = \frac{29}{10} = 2.9$, $ASL_{\text{败}} = \frac{39}{11}$ 。
2. 网络流最大流为 13。
3. 哈希表最终装填因子为 $\alpha = \frac{7}{11}$, 关键字 15 的探测路径为 1 -> 2 -> 5 -> 10。
4. 动态数组按 3 倍扩容时, 单次插入的均摊时间复杂度为 $O(1)$ 。
5. AOE 图中从 a 到 j 的最长路径长度为 21, 关键路径有两条:
 $a \rightarrow c \rightarrow e \rightarrow f \rightarrow g \rightarrow j$ 与 $a \rightarrow d \rightarrow h \rightarrow i \rightarrow j$ 。
6. 最少水盆问题应采用从左到右、优先放右侧空地的贪心策略。
7. 最长公共子串的动态规划递推为:
 $X[i] = Y[j]$ 时取左上角加 1, 否则清零; 时间复杂度为 $O(mn)$ 。

详细解析

一、选择题

1. D

解析：

由先序序列 ABDECFG 和中序序列 DBEACFG 可还原二叉树。根结点为 A。在中序序列中，A 左边的 DBE 构成左子树，右边的 CFG 构成右子树。继续还原可得：A 的右孩子为 C，C 的右孩子为 F，F 的右孩子为 G。在森林转换为二叉树的孩子兄弟表示中，一棵树的根结点的右孩子表示该森林中下一棵树的根结点。因此，从根结点 A 沿右链可得到 A, C, F, G 四个根，对应森林中的 4 棵树，故选 D。

2. B

解析：

二叉排序树满足：左子树上所有关键字小于根结点，右子树上所有关键字大于根结点。结合题图中 x 所在的位置，可以判断它必须大于 k_1 ，同时又小于 k_3 。因有 $k_1 < x < k_3$ ，故选 B。

3. A

解析：

删除大根堆的堆顶时，先用最后一个元素补到堆顶，再向下调整。第一次删除后：原堆顶 30 删除，用末尾元素 8 补到根处，再与较大孩子比较并下沉，可得新堆为 {25, 15, 20, 8, 10, 18, 12}。第二次删除时，再删除堆顶 25，用末尾元素 12 补到根处并向下调整。先与较大孩子 20 交换，再继续下沉，得到层序序列 {20, 15, 18, 8, 10, 12}。这与选项 A 一致，故选 A。

4. C

解析：

在 AVL 树中插入关键字 35 后，需要先按二叉排序树的规则完成插入，再检查从插入点向上的各祖先结点是否失衡。结合题图中的原始结构可知，失衡后经过相应的旋转调整，新的根结点变为 40。因此应选 C。

5. B

解析：

先根据权值构造 Huffman 树。权值最小的两个结点先合并：1 与 2 合并成 3；再将 3 与 7 合并成 10；再与另一个 10 合并。由此可以得到各叶子结点编码长度分别为： a 、 b 的编码长度为 4， c 的编码长度为 3， d 、 e 、 f 的编码长度为 2。所以：选项 A 正确，因为 d, e, f 编码长度相同；选项 B 错误，因为 c 的编码长度为 3，而 d, e 的编码长度为 2；选项 C 正确，因为 Huffman 树是严格二叉树，若叶子结点数为 n_0 ，则总结点数为 $2n_0 - 1$ ，即“总结点数加 1 等于叶子结点数的 2 倍”；选项 D 也正确，因为 Huffman 与 Prim 都体现了逐步做出局部最优选择的贪心思想。因此不正确的是 B。

6. B

解析：

对 0/1 背包问题而言，动态规划可以求最优解，分支限界法也可以求最优解，因此 A、C 的说法成立。但“直接采用贪心算法就一定能得到最优解”是错误的。0/1 背包中每件物品只能取或不取，不能像分数背包那样切分，因此按价值最大、重量最小或单位价值最大等单一贪心准则，都可能构造出反例，不能保证全局最优。选项 D 描述的是常见近似算法思想，可视为正确说法。故错误的是 B。

7. C

解析：

孩子兄弟表示法中，每个结点通常只保存“第一个孩子”和“下一个兄弟”两个指针。这种表示便于访问某结点的孩子和兄弟，但如果只知道某结点地址，要找它的双亲，一般需要从树根出发进行查找，并不能在 $O(1)$ 时间内直接得到。因而选项 C 错误。其余说法中：选项 A 正确，因为共有 10 个结点，除根结点外其余 9 个结点都需要记录双亲；选项 B 正确，因为孩子表示法天然便于列出一个结点的所有孩子；选项 D 也是常见结论。故选 C。

8. A

解析：

比较两个函数的增长速度，最直接的方法是看它们的比值： $\frac{f(n)}{g(n)} = \frac{3n^2+3n\log n+100}{n^2\log n}$ 。化简后可写为 $\frac{3}{\log n} + \frac{3}{n} + \frac{100}{n^2\log n}$ 。当 $n \rightarrow \infty$ 时，上式各项都趋于 0，所以 $\frac{f(n)}{g(n)} \rightarrow 0$ 。这说明 $f(n)$ 的增长速度严格慢于 $g(n)$ ，因此 $f(n)$ 一定是 $O(g(n))$ ，但不是 $\Theta(g(n))$ 。故正确说法为 A。

9. B

解析：

A 算法中，评价函数通常写作 $f(v) = g(v) + h(v)$ ，其中 $g(v)$ 是从起点到当前结点的实际代价， $h(v)$ 是从当前结点到目标结点的启发式估计代价。为保证 A 的最优性，启发函数 h 通常应满足“乐观估计”，也就是不高估从当前结点到目标结点的实际最短代价，因此 D 是正确的。按本题采用的方法， h 取当前结点所有出边权值中的最小值。因而 $h(v_5) = \min(2, 5) = 2$ ，所以 A 正确。对于 v_2 ，由起点到它的最短已知代价为 $g(v_2) = \min(7, 3+2) = 5$ ，同时 $h(v_2) = 6$ ，所以 $f(v_2) = g(v_2) + h(v_2) = 5 + 6 = 11$ ，并不是 13。因此不正确的是 B。按题图数据继续检查，其他选项可成立，所以应选 B。

10. C

解析：

逐项判断如下：第 1 项错误。希尔排序确实是不稳定的，但其最坏时间复杂度通常不能简单写成 $O(n \log n)$ ；第 2 项正确。归并排序稳定，辅助空间复杂度为 $O(n)$ ，从空间使用角度看确实不如堆排序；第 3 项正确。快速排序不稳定，在划分极不平衡时会退化为 $O(n^2)$ ；第 4 项错误。堆排序本身不稳定这一点是对的，但建立初始最大堆的时间复杂度应为 $O(n)$ ，不是 $O(n \log n)$ ；第 5 项正确。基数排序的定义或常见描述是成立的。因此正确的三项为 2、3、5，故选 C。

二、填空题

1. 24

解析：

题目说明矩阵下标从 1 开始，而压缩数组下标从 0 开始，这一点在计算时要特别注意。对称矩阵按下三角部分按行优先压缩存储时，第 i 行一共存储 i 个元素，因此在元素 $A[i][j]$ 之前，已经存放了前 $i-1$ 行的全部下三角元素，以及第 i 行中它前面的 $j-1$ 个元素。对于 $A[7][4]$ ，前 6 行共有 $1+2+3+4+5+6=21$ 个元素；在第 7 行中， $A[7][4]$ 前面还有 $A[7][1], A[7][2], A[7][3]$ 共 3 个元素。因而它在一维数组中的下标为 $21+3=24$ 。

2. 82

解析：

设叶子结点数为 n_0 。树中各类非叶子结点数分别为：20 个 4 度结点、10 个 3 度结点、1 个 2 度结点、10 个 1 度结点。因此非叶子结点总数为 $20+10+1+10=41$ 。由树的性质可知，整棵树的边数等于各结点度数之和，也等于总结点数减 1。所以边数为 $4 \times 20 + 3 \times 10 + 2 \times 1 + 1 \times 10 = 122$ 。于是总结点数为 $122+1=123$ 。因而叶子结点数为 $123-41=82$ 。

3. $O(\sqrt{n})$

解析：

程序中变量 i 依次取奇数 $1, 3, 5, 7, \dots$ ，每执行一次循环，就把当前奇数加到 $\sum$ 上。设循环执行了 t 次，则 $\sum = 1 + 3 + 5 + \dots + (2t-1)$ 。而前 t 个奇数之和等于 t^2 ，所以循环执行 t 次后有 $\sum = t^2$ 。循环终止条件是 $\sum \geq n$ ，因此有 $t^2 \geq n$ ，也就是 $t \approx \sqrt{n}$ 。所以循环次数与 $\sqrt{n}$ 同阶，时间复杂度为 $O(\sqrt{n})$ 。

4. 例如：a, b, d, e, f, c

解析：

由于题目没有规定访问某个顶点的邻接点时必须按哪一种次序选择，因此该题答案不唯一。只要所写序列满足“从起点 a 出发，尽可能沿一条未访问路径向深处搜索，无法继续时再回溯”的深度优先遍历原则，就可以视为正确。例如： $a \rightarrow b \rightarrow d$ 后， d 无未访问邻接点，于是回溯到 b ，再访问 e ，接着访问 f ，最后到达 c ，因而可得到一个合法序列 a, b, d, e, f, c 。

5. $2^{k-1} \leq n \leq 2^k - 1$

解析：

这道题要先说明“深度”的记法，因为教材中常见两种口径。第一种口径把结点深度定义为“从根到该结点所经过的边数”，此时根结点深度为 0，根的孩子深度为 1。这一说法常见于算法教材中。第二种口径把根结点看作第 1 层，也就是把“深度”按层数从 1 开始计，此时根结点深度为 1，根的孩子深度为 2。国内不少数据结构教材采用这一习惯。本题答案采用第二种口径，也就是“根结点在第 1 层”。于是树的深度为 k ，就表示这棵完全二叉树一共有 k 层。完全二叉树的特点是：除最后一层外，其余各层都一定是满的；最后一层的结点连续集中在左侧。因此前 $k-1$ 层一定全部排满，它

们的结点总数为 $1 + 2 + 4 + \dots + 2^{k-2} = 2^{k-1} - 1$ 。第 k 层至少有 1 个结点，因为整棵树深度已经达到 k ；第 k 层至多有 2^{k-1} 个结点，因为第 k 层最多只能排满这一层。所以整棵树的结点总数最少为 $(2^{k-1} - 1) + 1 = 2^{k-1}$ ，最多为 $(2^{k-1} - 1) + 2^{k-1} = 2^k - 1$ 。因而在本题采用的记法下，结点总数范围为 $2^{k-1} \leq n \leq 2^k - 1$ 。如果改用第一种口径，也就是根结点深度为 0，那么“深度为 k ”就意味着共有 $k + 1$ 层，此时结点总数范围应写成 $2^k \leq n \leq 2^{k+1} - 1$ 。

6. 长度为 3，深度为 3

解析：

广义表的长度指最外层所含元素的个数，而不是所有原子和子表总数之和。对于 $L = (a, ((b)), (c, (d, e)))$ ，最外层共有三个元素： a 、 $((b))$ 、 $(c, (d, e))$ ，所以长度为 3。再看深度。原子 a 的深度视为不再继续增加；子表 $((b))$ 的最深嵌套层数为 2；子表 $(c, (d, e))$ 的最深嵌套层数也为 2。整个广义表在这两者之外还有最外层一层，因此总深度为 3。

7. 一条可行路径为： $(4, 3) \rightarrow (3, 3) \rightarrow (3, 2) \rightarrow (2, 2) \rightarrow (1, 2)$ ；对应数值为： $2 \rightarrow 4 \rightarrow 6 \rightarrow 9 \rightarrow 10$

解析：

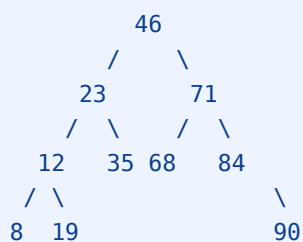
这里按“爬山法寻找更优结点”理解，即每一步都在当前点的四邻域中选择更大的相邻值继续前进，直到再也找不到更优邻点为止。起点 $(4, 3)$ 上的值为 2。它的相邻位置中，较优的候选点为 $(3, 3)$ ，对应值为 4，因此先移动到 $(3, 3)$ 。在 $(3, 3)$ 处，四邻域中更优的结点有 $(3, 2)$ ，其值为 6，继续移动到 $(3, 2)$ 。在 $(3, 2)$ 处，再向更优结点移动，可到达 $(2, 2)$ ，其值为 9。在 $(2, 2)$ 处，四邻域中更优的结点为 $(1, 2)$ ，其值为 10。到达 $(1, 2)$ 后，四邻域内已不存在比 10 更大的相邻值，因此算法停止。所以一条自然的移动路径为 $(4, 3) \rightarrow (3, 3) \rightarrow (3, 2) \rightarrow (2, 2) \rightarrow (1, 2)$ 。

三、大题

1.

答案：

1. 所构造的二叉排序树为：



2. 等概率成功查找的平均查找长度为

$$ASL_{\text{成}} = \frac{29}{10} = 2.9。$$

3. 等概率失败查找的平均查找长度为

$$ASL_{\text{败}} = \frac{39}{11}。$$

解析：

按关键字序列 46, 23, 71, 12, 35, 68, 84, 8, 19, 90 依次插入二叉排序树即可得到题中的结构。例如, $23 < 46$, 所以插到根的左侧; $71 > 46$, 插到根的右侧; $12 < 46$ 且 $12 < 23$, 所以成为 23 的左孩子, 后续结点依此类推。求成功查找平均查找长度时, 把每个结点所在层数作为查找长度。该树中: 第 1 层有 1 个结点; 第 2 层有 2 个结点; 第 3 层有 4 个结点; 第 4 层有 3 个结点。因而 $ASL_{\text{成}} = \frac{1 \times 1 + 2 \times 2 + 4 \times 3 + 3 \times 4}{10} = \frac{29}{10} = 2.9$ 。求失败查找平均查找长度时, 要统计查找落到空指针位置时所经历的比较次数。该树共有 10 个结点, 因此共有 11 个空链域。若按“比较到其双亲结点即停止”计, 则这些失败位置对应的查找长度总和为 39, 所以 $ASL_{\text{败}} = \frac{39}{11}$ 。

2.

答案:

汇点 t 的最大接受流量为 13。

一组可达到最大流的流量分配方案可写为: $f(s, v_1) = 6$, $f(s, v_3) = 7$, $f(v_3, v_1) = 1$, $f(v_1, v_2) = 4$, $f(v_1, v_4) = 3$, $f(v_3, v_4) = 6$, $f(v_2, t) = 4$, $f(v_4, t) = 9$ 。

解析:

要求最大流, 核心思路是不断在余图中寻找增广路径, 直到再也找不到从源点到汇点的增广路径为止。从结论上看, 汇点 t 的入边容量之和为 $c(v_2, t) + c(v_4, t) = 4 + 9 = 13$, 因此最大流不可能超过 13。另一方面, 上面给出的一组流量分配确实能够使进入 t 的总流量达到 $4 + 9 = 13$, 并且各条边上的流量都不超过容量, 同时满足除源点和汇点外各中间结点的流量守恒。由于“上界”与“可实现的流量值”恰好相等, 所以可以确定最大流就是 13。这类题在草稿中通常可以借助流图和余图逐步验证。

3.

答案:

1. 最终散列表如下:

0	1	2	3	4	5	6	7	8	9	10
-	8	9	3	-	19	6	20	-	-	15

2. 装填因子为

$$\alpha = \frac{7}{11}。$$

3. 查找关键字 15 时的探测路径为:

1 -> 2 -> 5 -> 10。

解析:

先按哈希函数 $H(k) = k \bmod 7$ 计算初始地址: $H(8) = 1$, 所以 8 放在地址 1; $H(19) = 5$, 所以 19 放在地址 5; $H(3) = 3$, 所以 3 放在地址 3; $H(9) = 2$, 所以 9 放在地址 2; $H(6) = 6$, 所以 6 放在地址 6。对 15 而言, $H(15) = 1$, 但地址 1 已被 8 占用, 于是采用平方探测: $(1 + 1^2) \bmod 11 = 2$, 仍冲突; $(1 + 2^2) \bmod 11 = 5$, 仍冲突; $(1 + 3^2) \bmod 11 = 10$, 空闲, 因此 15 放到地址 10。对 20 而言, $H(20) = 6$, 地址 6 已有 6, 继续探测: $(6 + 1^2) \bmod 11 = 7$, 地址 7 空闲, 因此 20 放到

地址 7。散列表长度为 11，共存入 7 个关键字，所以装填因子为 $\alpha = \frac{7}{11}$ 。查找 15 时，探测过程与其插入时一致，因此路径就是 1 -> 2 -> 5 -> 10。

4.

答案：

单次插入操作的均摊代价为常数级，均摊时间复杂度为 $O(1)$ 。

解析：

采用聚集分析法来说明最直观。数组初始容量为 1，每次扩容变为原来的 3 倍，因此容量序列为 1, 3, 9, 27, ...。如果在前 n 次插入过程中一共发生了若干次扩容，那么每次扩容的主要代价来自把旧数组中的元素全部复制到新数组中。这些复制代价依次为 1, 3, 9, ..., 3^{t-1} ，其和为等比数列 $1 + 3 + 9 + \dots + 3^{t-1} = \frac{3^t - 1}{2}$ 。当进行了 n 次插入时，必然存在某个 t 使得 $3^{t-1} < n \leq 3^t$ 。因为 $3^t < 3n$ ，所以复制总代价小于 $3\frac{n}{2}$ 。再加上每次普通插入自身的代价 1，共 n 次，因此总开销小于 $n + 3\frac{n}{2} = 5\frac{n}{2}$ ，也就是前 n 次插入的总代价为 $O(n)$ 。用总代价除以操作次数 n ，可得单次插入的均摊代价为常数级，因此均摊时间复杂度为 $O(1)$ 。

5.

答案：

1. 邻接表可写为：

$a: (b, 4), (c, 5), (d, 6); b: (f, 4); c: (e, 1); d: (h, 9); e: (f, 3), (g, 4); f: (g, 4); g: (h, 1), (j, 8); h: (i, 2), (j, 3); i: (j, 4); j: \text{空}。$

2. 两个不同的拓扑序列例如：

$a, b, c, d, e, f, g, h, i, j$ 和 $a, c, b, d, e, f, g, h, i, j。$

3. 从顶点 a 到顶点 j 的最长路径长度为 21。

所有关键路径为： $a \rightarrow c \rightarrow e \rightarrow f \rightarrow g \rightarrow j$ 和 $a \rightarrow d \rightarrow h \rightarrow i \rightarrow j。$

解析：

邻接表就是把每个顶点的出边依次列出，因此根据题图逐点读取即可得到第 1 问答案。写拓扑序列时，只要满足“每条有向边的起点都出现在终点之前”即可，因此同一个 DAG 往往不止一个拓扑序列。对第 3 问，可以直接比较从 a 到 j 的各条主要路径长度。其中： $a \rightarrow c \rightarrow e \rightarrow f \rightarrow g \rightarrow j$ 的长度为 $5 + 1 + 3 + 4 + 8 = 21$ ； $a \rightarrow d \rightarrow h \rightarrow i \rightarrow j$ 的长度为 $6 + 9 + 2 + 4 = 21$ 。这两条路径长度相同，且都达到从 a 到 j 的最大值，因此它们都是关键路径。其余从 a 到 j 的可行路径长度都小于 21，所以不会成为关键路径。

6.

答案：

1. 输入 $[0, 1, 1, 0, 1]$ 时，输出为 2；输入 $[1, 1, 0]$ 时，输出为 -1。

2. 贪思路：

从左到右扫描数组。当遇到一只当前还没有水可喝的小猫时，优先在它右侧相邻空地放置水盆；若右侧不能放，再考虑左侧相邻空地；若两侧都无法放置，则不存在可行方案，返回 -1。

3. 伪代码如下：

```
count <- 0
bowl[1..n] <- false
for i <- 1 to n:
  if A[i] = 1:
    if i > 1 and bowl[i-1] = true:
      continue
    else if i < n and A[i+1] = 0:
      bowl[i+1] <- true
      count <- count + 1
    else if i > 1 and A[i-1] = 0 and bowl[i-1] = false:
      bowl[i-1] <- true
      count <- count + 1
    else:
      return -1
return count
```

解析：

这道题的关键是说明为什么“优先放在当前小猫右边”是合理的。当我们从左向右扫描时，如果当前位置是一只还没有被满足的小猫，那么它必须依靠左边或右边的相邻空地放置水盆。若右边可以放水盆，优先放在右边不会比放在左边更差，因为右边的水盆既能满足当前小猫，还有机会继续满足右边后续位置上的小猫；而把水盆放在左边，通常只能服务于已经扫描过的位置，向后的覆盖能力较弱。这样逐步做局部最优选择，最终可以得到全局最少的水盆数。例如输入 $[0,1,1,0,1]$ 时，扫描到第一个 1 可在其左边或右边考虑，但按贪心策略放置后，最终至少需要两个水盆；而输入 $[1,1,0]$ 时，最左边的小猫左侧没有空地，右侧又是小猫，因此无法安排使所有小猫都喝到水，答案为 -1。伪代码中的 bowl 数组用来标记哪些空地已经放了水盆，从而避免重复计数。

7.

答案：

1. 递推关系：

设 $dp[i][j]$ 表示以 $X[i]$ 和 $Y[j]$ 结尾的最长公共子串长度，则当 $X[i] = Y[j]$ 时， $dp[i][j] = dp[i-1][j-1] + 1$ ；当 $X[i] \neq Y[j]$ 时， $dp[i][j] = 0$ 。

2. 伪代码：

```
maxLen <- 0
endPos <- 0
for i <- 0 to m:
  dp[i][0] <- 0
for j <- 0 to n:
  dp[0][j] <- 0

for i <- 1 to m:
```



```

for j <- 1 to n:
  if X[i] = Y[j]:
    dp[i][j] <- dp[i-1][j-1] + 1
    if dp[i][j] > maxLen:
      maxLen <- dp[i][j]
      endPos <- i
  else:
    dp[i][j] <- 0

```

输出最长公共子串 X[endPos - maxLen + 1 .. endPos]

输出 maxLen

3. 时间复杂度为

$O(mn)$ 。

解析：

本题与最长公共子序列很像，但区别在于这里要求的是“子串”，也就是必须连续。因此状态定义成“以 $X[i]$ 和 $Y[j]$ 结尾的最长公共子串长度”最合适。如果 $X[i] = Y[j]$ ，说明当前字符可以接在前一个公共子串后面，于是长度等于左上角状态再加 1；如果 $X[i] \neq Y[j]$ ，由于连续性被打断，以这两个位置结尾的公共子串长度只能重新归零，所以 $dp[i][j] = 0$ 。整个状态表共有 $m \times n$ 个位置，每个位置只需常数时间完成转移，因此总时间复杂度为 $O(mn)$ 。若完整保存整个二维状态表，则空间复杂度也是 $O(mn)$ ；如果只要求长度而不必恢复具体子串，还可以进一步做空间优化。